

DATABASE SPECIFICATIONS

Myki Card Transportation System Database
Jordan Chiou,
jungdechou@gmail.com

School of Computing Technologies, RMIT University

March, 2025

Revision Sheet

Release No.	Date	Revision Description
Rev. 0	26/03	Complete Chapter 1 – General Information
Rev. 1	05/04	Refine chapter 1 and Building chapter 2
Rev. 2	13/04	Refine chapter 1 and Complete chapter 2
Rev. 3	18/04	Refine chapter 2
Rev. 4	19/04	Refine chapter 2 and Building chapter 3
Rev. 5	20/04	Refine chapter 2 and Complete chapter 3

DATABASE SPECIFICATIONS

TABLE OF CONTENTS

	<u>Page #</u>
1.0 GENERAL INFORMATION	1
1.1 Purpose.....	1
1.2 Scope.....	1
1.3 System Overview	1
1.4 Project References	3
1.5 Acronyms and Abbreviations	4
1.6 Points of Contact.....	4
1.6.1 Information.....	4
1.6.2 Coordination.....	4
1.6.3 Additional Points of Contact.....	4
1.6.4 Data Owners.....	4
2.0 DATABASE IDENTIFICATION AND DESCRIPTION.....	1
2.1 Naming Conventions	1
2.2 Database Identification	2
2.3 Systems Using the Database	2
2.4 Relationship to Other Databases	3
2.5 Schema Information.....	3
2.5.1 Description	3
2.5.2 Physical Design.....	5
2.5.3 Physical Structure.....	9
2.6 Data Dictionary	12
2.7 Special Instructions	13
3.0 DATABASE ADMINISTRATIVE INFORMATION	1
3.1 Responsibility	1
3.2 System Information.....	1
3.2.1 Database Management System (DBMS) Configuration.....	1
3.2.2 Hardware Configuration.....	1
3.2.3 Database Software Utilities.....	1
3.2.4 Support Software Available for Maintaining Database	2
3.2.5 Security	3
3.3 Storage Requirements	3
3.4 Recovery	3
3.5 Partition/File Information	4
3.5.1 Content.....	4
3.5.2 Description	4
3.5.3 Partition/File Interdependencies.....	6
3.6 Database Interfaces	7
3.6.1 Description of Operational Implications.....	7
3.6.2 Description of Data Transfer Requirements	7
3.6.3 Description of Formats of Data.....	7

3.7 Error Handling	7
--------------------------	---

1.0 GENERAL INFORMATION

GENERAL INFORMATION

1.1 Purpose

This document defines the logical and physical design specifications for the **Myki Card Transportation System Database**. The primary purpose of this specification is to replicate the core functionality of the existing Myki system while focusing on optimising its backend database performance, including:

- **Touch-on / touch-off trip recording**
- Fare calculation and capping rules
- Transaction tracking (fare deductions and top-ups)
- Vehicle and station tracking for determining travel locations
- System extensibility for various transport modes (tram, train, bus)

This project aims to address the key database performance bottlenecks identified in the current system by applying advanced database design principles, such as index tuning, partitioning strategies, and effective storage management. Additionally, this specification estimates long-term storage requirements for the next 10 years based on projected usage growth and data accumulation patterns.

The proposed database model is designed for implementation in a Microsoft SQL Server environment and serves as a reference for database administrators, system developers, and other stakeholders involved in its development and maintenance.

1.2 Scope

This database specification defines the design and structure of the backend relational data storage for the Myki Card System, with a primary focus on supporting public transport operations within the metropolitan area for full-fare and concession passengers.

In particular, this specification covers:

- Modelling of Myki cards and customer account information
- Capturing touch-on and touch-off events through scanning devices
- Recording trip histories along with relevant travel metadata
- Storing transaction records for fare deductions, top-ups, and refund.
- Integrating vehicles, routes, and stop stations to determine precise device location data

- Deriving boarding and alighting stations based on vehicle movement logs and GPS tracking data

The scope of this specification explicitly excludes regional transport logic and any integration with external fare enforcement or financial settlement systems. These components may be considered for future system enhancements.

This defined scope provides a robust foundation for capturing and processing core Myki usage scenarios while ensuring the database design is extensible for future operational requirements.

1.3 System Overview

This system represents a simplified smartcard fare collection platform similar to the Myki card system used in Victoria's metropolitan public transport network. Its objective is to serve as a proof-of-concept for implementing advanced relational database design and optimisation techniques in a public transport setting.

The database design emphasises key operations such as touch-on/touch-off event processing, trip logging, fare calculation, transaction recording, and vehicle location data integration. This system creates a structured environment for demonstrating database modelling principles, performance tuning methods, and storage optimisation techniques.

- Responsible organisation:

Not Applicable—This system was developed as part of an academic coursework project within RMIT University for educational purposes. No formal organisation is responsible for production operations.

- System name or title:

Myki Card Transportation System Database

- System code:

Not Applicable

- System category:

Major application — The system performs defined functions (touch-on, trip tracking, fare deduction) with identifiable logical components and business rules. This system is not a general-purpose support system.

- Operational status:

Under Development — This system is in the design and prototyping phase for academic demonstration and is not intended for production deployment.

- System environment and special conditions:

This database system is developed within a controlled educational environment. Key assumptions and design conditions include:

- Target Relational Database Management System (RDBMS): Microsoft SQL Server
- Assumes integration with simulated GPS and log-based inputs for vehicle movement tracking
- Focus is limited to backend database design; no front-end interface, API integration, or external payment processing components are within the scope of this system.

1.4 Project References

The following external resources were used to inform and support the design of this database specification:

1. **Public Transport Victoria (PTV) GTFS Feed – Vehicle Types**
URL: <https://data.ptv.vic.gov.au/downloads/GTFSReleaseNotes.pdf>
2. **Train Service Passenger Counts – Data Structure Reference**
URL: <https://discover.data.vic.gov.au/dataset/train-service-passenger-counts/resource/162887ef-1dba-4d9b-83bd-baee229229c6>
3. **TramTracks Real-Time Tram Location Feed**
URL: <https://discover.data.vic.gov.au/dataset/tram-tracks/resource/60dc379e-1ac9-4e3c-9dca-c428f5c014f0>
4. **Myki Fare Structure – Metropolitan Zones and Default Fares**
URL: <https://www.ptv.vic.gov.au/tickets/fares/metropolitan-fares/#defaultfares>
5. **Victoria Population projection:**
URL: https://www.planning.vic.gov.au/_data/assets/pdf_file/0022/703453/DTP0552-Victori-in-Future-2023-report.PDF
6. **Method of travel to work:**
<https://www.abs.gov.au/census/guide-census-data/census-dictionary/2021/variables-topic/transport/method-travel-work-mtwp>
7. **PTV Annual report:**
<https://www.vic.gov.au/dtp-annual-report>

1.5 Acronyms and Abbreviations

Acronyms or Abbreviations	Description
DB	Database
DBMS	Database Management System
RDBMS	Relational Database Management System
PTV	Public Transport Victoria
GPS	Global Positioning System
SQL	Structured Query Language
T-SQL	Transact-SQL (Microsoft SQL Server extension to SQL)
ERD	Entity-Relationship Diagram
RMIT	Royal Melbourne Institute of Technology
udf	User Defined Function
usp	User Stored Procedure

1.6 Points of Contact

1.6.1 Information

Name: Jordan Chiou

Role in this project: Database Designer

Contact: jungdechiou@gmail.com / s4068959@student.rmit.edu.au

1.6.2 Coordination

Not Applicable — No additional coordination roles are defined for this project.

1.6.3 Additional Points of Contact

Not Applicable — No other contributors or stakeholders are involved in this project.e

1.6.4 Data Owners

Data Owner	Responsibility
Jordan Chiou	Responsible for database design, data modelling, and overall data ownership for the project.

2.0 DATABASE IDENTIFICATION AND DESCRIPTION

DATABASE IDENTIFICATION AND DESCRIPTION

2.1 Naming Conventions

1. Logical Naming Conventions:

Logical names are used in entity-relationship (ER) diagrams and conceptual discussions.

- **Entities** (tables) use **PascalCase**, singular form.
 - Example: MykiCard, Trip, VehicleRun, StopStation
- **Attributes** (columns) use **snake_case**, descriptive and self-explanatory.
 - Example: card_id, touch_on_time, vehicle_type
- Primary keys are named as entity_name_id or abbreviated as xxx_id followed with {PK} symbol.
 - Example: trip_id {PK}, route_id{PK}
- Foreign keys are not explicitly marked in this ER diagram but are named identically to the referenced primary key.
 - Example: card_id in Trip references MykiCard.card_id
- Enumerated values (e.g. CardType, VehicleType) are shown as fixed values or look-up tables.

2. Physical Naming Conventions (SQL Server)

These conventions apply to the actual implementation of the schema in SQL Server.

- **Schema prefix:** All tables are implemented under the default dbo schema, and the “dbo.” prefix is used in all fully-qualified object references.
 - Example: dbo.VehicleRun, dbo.Trip
- **Table names:** Use PascalCase without pluralisation, consistent with the logical model.
- **Column names** use lowercase with underscores (snake_case), consistent with the logical model.
- **Constraint names:** Follow a descriptive format with standard prefixes. These reflect the associated table and attribute(s).
 - Example: CK_CardTransaction_Amount, FK_Trip_CardID
- **Index names:** Use the prefix IX_ followed by the table name, columns, or index purpose.
 - Example:

- IX_Trip_Card_TouchOn – for fast lookups by card and time
- IX_Trip_UnfinishedTripCheck – for checking active trips during touch-on
- **Data types:** Chosen based on the attribute's nature and expected range of values.
 - Example: DECIMAL(10,2) for monetary values, DATETIME2(0) for timestamps, and VARCHAR(100) for textual fields

2.2 Database Identification

The database defined in this specification is logically referred to as:

| MykiTransportDB

This logical name denotes the intended schema design for simulating the Myki card fare collection system in a metropolitan public transport network. However, the actual database name on the instructional Microsoft SQL Server is limited to the student ID s4068959, representing the real Database Name on this database design's SQL server. The schema is set to dbo, and all table-level design and naming conventions follow the logical data model outlined in this document. The schema is implemented using a single consistent structure, with each table name serving as a unique, descriptive identifier. Prefixes or abbreviations are avoided to maximise readability and traceability.

Database Code Name

- **Logical Database Name:** MykiTransportDB
- **Actual Database Name(SQL Server):** s4068959

Table Identifier:

Table Name	Description
CustomerAccount	Stores user information for registered customers
MykiCard	Represents each issued card, balance, and type
CardType	Enumerated card fare types (full-fare, concession, etc.)
MykiPass	Record of Myki pass purchased by users with start date and expiry date
FareType	Lookup table for types of fare (default_fare, free_2hours, etc.)
Trip	Captures travel journeys initiated by card taps
CardTransaction	Logs monetary operations such as fare deductions or top-ups
Scanner	Represents touch-on/off scanning devices
DeviceLocation	Abstract link between scanner and physical location
StopStations	Fixed locations where passengers board or alight
Route	Defines public transport routes

RouteStop	Mapping of ordered stop points for each route
Vehicle	Transport units (tram, bus, train)
VehicleType	Lookup for types of vehicles
VehicleRun	A scheduled or real-time instance of a vehicle operating a route
VehicleStopLog	System-generated record of vehicle arrival/departure at stations
VehicleRealTimeLog	Raw GPS location log for tracking vehicle movement
Calendar	Aligned with the national calendar to decide which day is a weekend and a holiday for the pricing system

Within the scope of the Mykitransportdb schema, all table names are unique and self-descriptive. The naming conventions are applied to ensure consistency, clarity, and ease of maintenance across database development and operation activities.

2.3 Systems Using the Database

Not Applicable — This database is designed for academic demonstration purposes only. No external systems or applications are connected to it. During development and testing, all operations and data access are performed directly within the Microsoft SQL Server environment.

2.4 Relationship to Other Databases

Not Applicable — This database operates as a standalone system for academic demonstration purposes. There is no integration with external databases or data exchange with other systems. All data required for the Myki Card Transportation System is self-contained within the MykiTransportDB schema.

Design Note:

If the system were to be extended for real-world deployment, it is recommended to separate database systems based on business domains to improve scalability, maintainability, and system performance. For example:

- A dedicated *Myki Payment Database* to handle card management, fare calculation, and transaction processing (*Blue components in the ERD*).
- A separate *Vehicle Operation Database* to manage real-time vehicle location tracking and operational logs (*Green components in the ERD*).

2.5 Schema Information

2.5.1 Description

The database is structured as a single schema logically named *myki*, representing a modular and normalised relational structure. This schema includes all entity, reference, and operational tables to simulate a Myki card-based fare management system in a metropolitan transport context.

In the instructional Microsoft SQL Server environment, the implementation uses the default schema name `dbo`. To maintain conceptual clarity, all the logical design adheres to the MykiTransportDB naming convention throughout this document.

2.5.1.1 Schema Name

- Logical schema name: **myki**
- Actual schema name (SQL Server): **dbo**
- Target RDBMS: **Microsoft SQL Server**

2.5.1.2 Sub-schemas

- Not Applicable — the system is implemented as a single schema without physical sub-schemas.

2.5.1.3 File Type / Data Model

- The database follows a **relational data model** and generally adheres to **Third Normal Form (3NF)** principles to minimise redundancy and ensure data integrity.
- However, selected **controlled denormalisation** is intentionally applied in specific tables to improve performance, like the `DeviceLocation` table includes a redundant field `current_stop_station_id` to enable faster real-time access.
- All tables are defined using **SQL Data Definition Language (DDL)** statements compatible with Microsoft SQL Server.
- Constraints are applied at both table and column levels to enforce data integrity, including **primary keys**, **foreign keys**, and **check constraints** (e.g., enforcing non-negative transaction amounts and logical date relationships).
- Additional T-SQL stored procedures or functions are implemented for specific business logic (e.g., touch-on, touch-off, checkFareType operations, etc.)

2.5.1.4 Access Control Keys

- Not Applicable — User-level access control is not implemented in this academic prototype. All database operations are executed within a trusted environment, and no role-based access management is required at this stage.

2.5.1.5 Concurrency and Locking

- The default concurrency control mechanisms provided by Microsoft SQL Server are used. No custom locking logic or isolation level overrides have been implemented.

- The system relies on SQL Server's default Read Committed isolation level, which is sufficient for this version's operational requirements. Alternative strategies (e.g., snapshot isolation, row-level locking hints) may be evaluated if future versions require higher transaction throughput or consistency guarantees.

2.5.1.6 Data Name Mapping

- All table and column names follow the naming conventions defined in *Section 2.1*. Table names use **PascalCase** and column names use **snake_case** to maintain clarity, readability, and consistency across the schema.
- In Microsoft SQL Server, all objects are defined under the default schema dbo. As such, fully qualified table references follow the dbo.TableName format (e.g., dbo.Trip, dbo.MykiCard).
- User-defined functions follow the prefix convention udf_FunctionName (e.g., udf_IsHoliday) to clearly distinguish them from built-in functions.
- Stored procedures follow the prefix usp_ProcedureName (e.g., usp_TouchOn) to reflect their procedural nature and usage context.
- This version does not introduce **aliases, synonyms, or abstraction layers** in order to maintain schema simplicity and minimise interpretative overhead during development and testing.

2.5.1.7 Partition and File Limitations

- No table or index is horizontally sharded across multiple filegroups.
- The entire schema is hosted within a single logical database instance, under the default SQL Server configuration provided in the teaching environment.
- Due to platform restrictions, all objects are stored within the default PRIMARY filegroup, and custom filegroup allocation is not supported in this deployment.
- While selected high-volume tables are logically partitioned to improve query performance and support scalability, physical-level filegroup optimisation (e.g., I/O balancing, storage tiering, or multi-disk parallel access) is considered out of scope for this academic implementation.

2.5.1.8 Access Path Restrictions

- No materialised views, full-text indexes, or computed columns are defined in this version.
- A **logical view** (vwCustomerAccount) is implemented to support unified access to vertically partitioned data from CustomerAuth and CustomerProfile. This view is not indexed and is intended for convenience during data retrieval only.

- All other data access is performed through direct queries on base tables, using defined **primary and foreign key constraints**, along with **manually created indexes** optimised for touch-on/off and trip-tracking operations.

2.5.2 Physical Design

The physical design of the database represents the conceptual structure of the system as implemented through entity relationships.

Figure 2.1 below illustrates the complete Entity-Relationship Diagram (ERD) that defines the entities, relationships, and key constraints within the MykiTransportDB schema.

The ERD is logically divided into two thematic segments:

- **Blue section:** Represents the **Myki Card management logic**, including cards, accounts, transactions, trips, scanners, and passes. This is the **primary focus** of the database design.
- **Green section:** Represents **vehicle and route-related data**, which is included to support trip recording and fare calculation based on movement. This section is intentionally simplified, as it plays a supporting role to the card interaction logic.

In addition, several design assumptions and notes are outlined below to support the simplification or interpretation of certain data elements where explicit documentation from the real-world Myki system is unavailable.

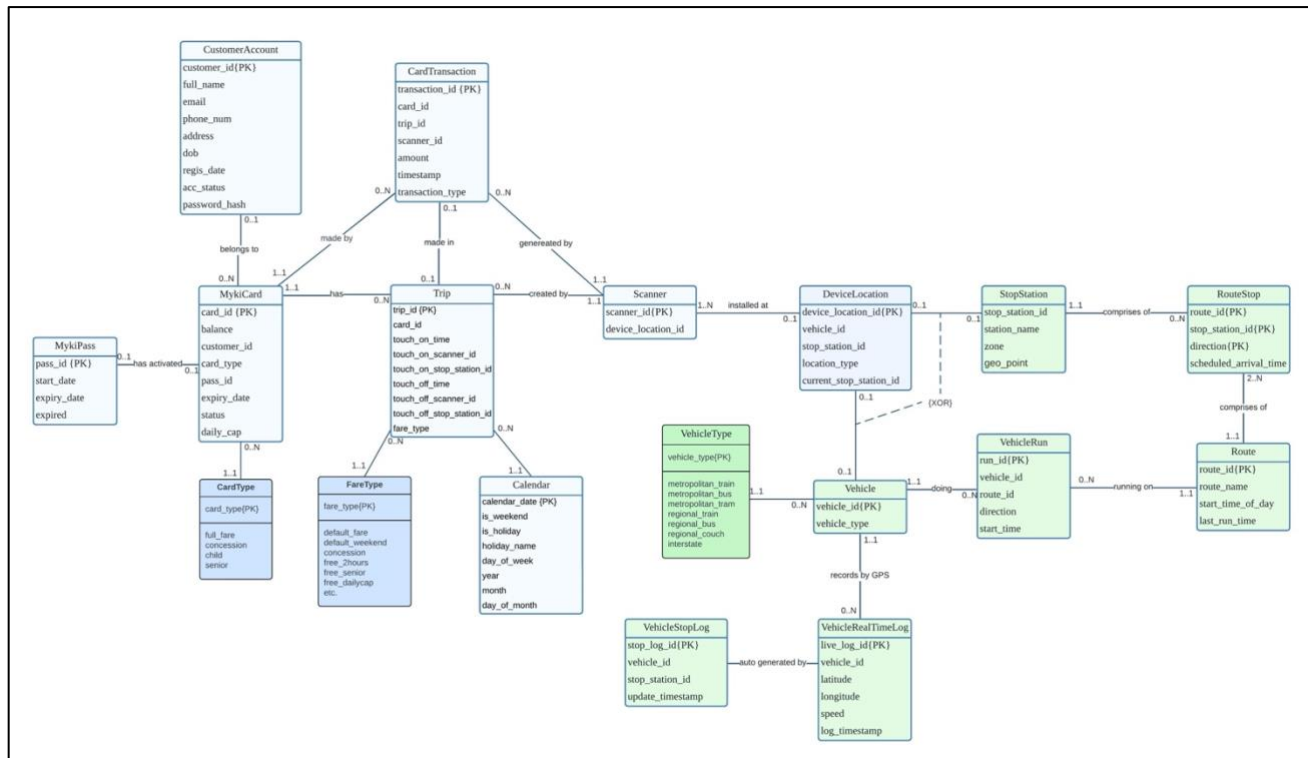


Figure 2.1

2.5.2.1 Notes:

1. MykiCard Table:

- **MykiCard records may exist without an associated customer account.** These represent anonymous cards.
- Card balances can go down to a maximum overdraft of **−10 AUD**.
- The status field and expiry_date should remain consistent — cards with status = 0 (expired) should have an expiry date earlier than the current date.

2. Calendar Table:

- This table is used by the CardTransaction logic to differentiate between weekday and weekend fare policies.
- This table is manually constructed for testing purposes and does **not** pull data from an external or government calendar API.

3. Scanner Table:

- Each **Scanner** is assigned to **one** DeviceLocation, while a single DeviceLocation (e.g., a tram or a station) may host **multiple** scanners.
- **Top-up machines are treated as scanners** for simplicity. However, the specific logic of the top-up process is outside the scope of this project.

4. DeviceLocation Table:

- In the **DeviceLocation** table, the attributes vehicle_id and stop_station_id are **mutually exclusive** — a scanner cannot be located in both a vehicle and a fixed stop.
- For **online transactions**, the location_type is set to "online"; for physical interactions, it is set to "physical".

5. This design does **not explicitly differentiate** between transport modes (e.g., tram vs. train) in terms of stop or route logic. The stop and route structure is simplified to test zone-based fare calculations rather than implement a full transport network simulation.

2.5.2.2 Assumptions & Justifications

1. All Vehicles Require Touch-On and Touch-Off

Assumption: All public transport vehicles in the system, including trams, require both touch-on and touch-off actions.

Changes:

- This deviates from the current real-world tram system, which does not enforce touch-off.
- Under this assumption, fare deductions only occur upon touch-off.

- The system maintains the current *forgot-to-touch-off* policy, where a full “Zone 1+2 2-hour” fare is charged when passengers forget to touch off.

Benefits:

- Establishes a **uniform usage pattern** for all Myki cards, improving user understanding and system predictability.
- Enables PTV to collect **more granular trip data** for future analysis, especially regarding tram usage.
- Lays the groundwork for future transitions from zone-based pricing to **distance-based dynamic fare models**.

2. Real-Time GPS Logging Every 30 Seconds

Assumption: Each vehicle has a GPS device that sends location updates every 30 seconds. These logs are directly stored in the VehicleRealTimeLog table.

Justification: This simulates a common real-time vehicle tracking system used in intelligent transport infrastructure and provides accurate location snapshots for determining stop arrival patterns and trip routes.

3. Stop Log Generation Based on Real-Time Location

Assumption: The VehicleStopLog table is derived from real-time location data (VehicleRealTimeLog).

While not implemented in this version, the design assumes the system would:

- Compare GPS coordinates with known StopStation locations
- Use vehicle_id and vehicle_type to avoid misclassifying stop points (e.g., assigning a tram stop to a bus)
- Determine the nearest matching stop and route for each entry to decide which stop_station_id the vehicle belongs to.

Justification: This process mirrors real-world transit systems that geofence or match GPS coordinates to scheduled stop points for accurate trip recording.

4. Dynamic Update of DeviceLocation Based on Vehicle Stop

Assumption: The current_stop_station_id in the DeviceLocation table is dynamically updated based on the most recent VehicleStopLog entry for the associated vehicle if the stop_station_id has changed.

Justification:

- Improves real-time accuracy of location data during *touch-on/off* operations.
- Avoids expensive multi-table joins by allowing the scanner to reference the current location directly from DeviceLocation.

5. Pass Expiry and Card-Pass Association Management

Assumption: A MykiCard is associated with a MykiPass via the `pass_id` field in the MykiCard table.

The design assumes a background process (not implemented) that:

- Periodically checks for expired passes
- Sets `expired = 1` in the MykiPass table when expiry is detected
- Sets the corresponding `MykiCard.pass_id = NULL` to indicate disassociation from an invalid pass

Justification:

- Provides a simple yet effective way to detect active pass usage on a card
- Keeps card and pass relationships clean without requiring complex joins or subqueries during fare evaluation

2.5.3 Physical Structure

This section of the *MykiTransportDB* database captures the actual implementation within Microsoft SQL Server. The deployed schema's concrete components, including partitioned tables, indexing strategy, and runtime data structure, are documented here. *Figure 2.2* below shows a graphical overview of the physical structure as implemented.

While the ERD in *Figure 2.1* illustrates the logical relationships among entities, the actual implementation may diverge due to database-level constraints such as partitioning, foreign key enforcement, and practical performance considerations.

For instance, the `Trip` table is partitioned by `touch_off_time`, as only completed trips (i.e., those with a non-placeholder `touch_off_time`) are eligible for fare calculation and archiving. As a result, `touch_off_time` must be included in the primary key. This, in turn, affects related tables such as `CardTransaction`, which must also include `touch_off_time` in their foreign key reference to maintain referential integrity under SQL Server's partitioning constraints.

Another example is the relationship between `Trip` and `Calendar`, which is modelled logically in the ERD but implemented as a function-based lookup. Instead of using an enforced foreign key, fare

calculation queries retrieve the relevant calendar attributes dynamically based on the current date. This design provides greater flexibility and reduces join complexity in the touch-on/touch-off process.

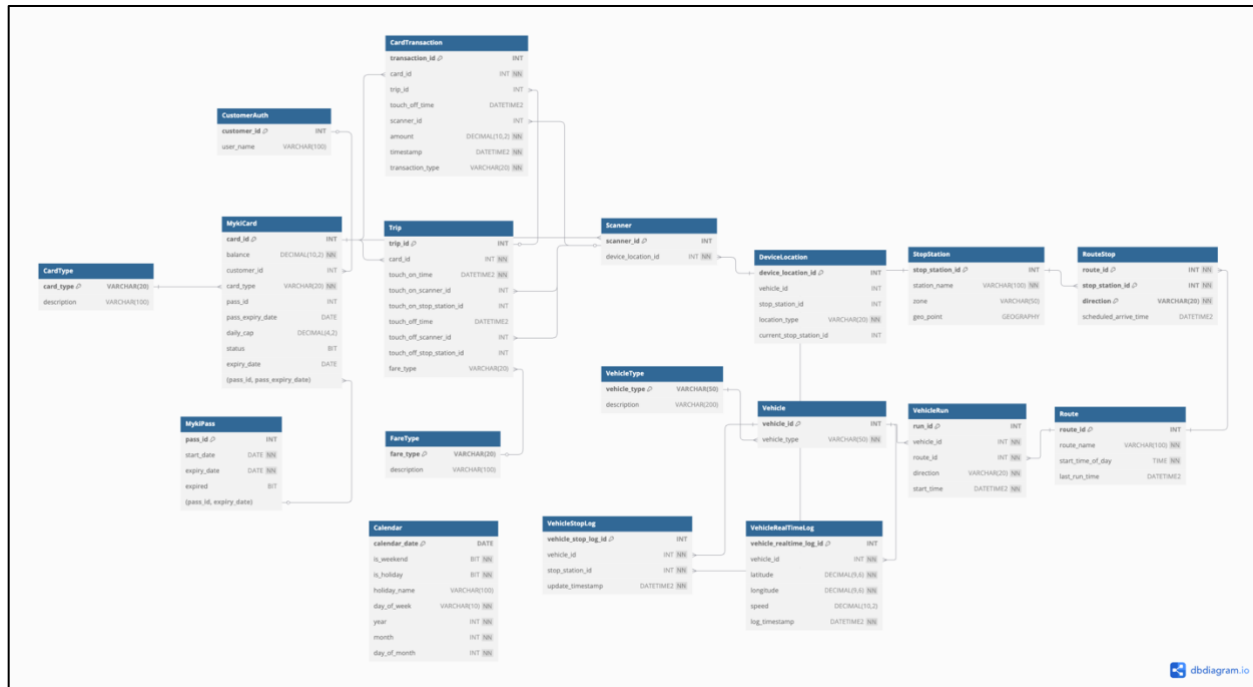


Figure 2.2

2.5.3.1 Partitioning Strategy

Horizontal partitioning has been implemented on selected high-volume tables using DATETIME2 fields to improve scalability and enhance query performance. A monthly range-based partitioning strategy is in place, executed through a partition scheme (ps_Monthly) and a corresponding partition function (pf_Monthly) that defines boundary ranges for each month.

The existing partition function covers 2024-02-01 to 2030-02-01 (exclusive), encapsulating six years of anticipated operational activity. This partitioning is applied to tables such as Trip, CardTransaction, VehicleStopLog, and VehicleRealTimeLog, which all manage rapidly growing time-series data.

Monthly partitioning offers a reasonable balance between query efficiency and the management overhead of partitions. Although daily partitioning might enhance write throughput for significantly large datasets, it has been intentionally dismissed to mitigate the risk of generating too many partitions. The excessive partitioning would substantially increase metadata overhead and complicate storage management in SQL Server.

Additionally, a secondary partition scheme (ps_Yearly) and a function(pf_Yearly) have been established to accommodate potential expansion for archival tables, such as MykiPass, that may benefit from broader yearly-based partitioning dates.

Table	Partition Key	Partition Strategy
Trip	touch_off_time	Monthly range partition
CardTransaction	timestamp	Monthly range partition
VehicleStopLog	update_timestamp	Monthly range partition
VehicleRealTimeLog	log_timestamp	Monthly range partition
MykiPass	expiry_date	Yearly range partition

Vertical partitioning is applied to the *CustomerAccount* table to optimise query performance and storage efficiency. This partitioning strategy separates frequently accessed authentication-related data from less frequently used customer profile information.

Specifically, the *CustomerAccount* table is vertically partitioned into the following two logical tables:

- *CustomerAuth*: Contains attributes required for user authentication and account management, including customer_id, email, password_hash, acc_status, and regis_date. This table is expected to be frequently accessed during login and account validation processes.
- *CustomerProfile*: Stores customer profile information that is less frequently accessed or updated, such as full_name, phone_num, address, and dob.

Table	Attributes
CustomerAuth	customer_id (PK), email, password_hash, acc_status, regis_date
CustomerProfile	customer_id (PK), full_name, phone_num, address, dob

To maintain logical consistency and support seamless data access, a **view** named vwCustomerAccount is implemented to provide a unified interface for querying vertically partitioned customer data. This view abstracts the underlying table separation, allowing external queries to interact with the combined customer record as if it were a single entity.

Additionally, a trigger named trg_vwCustomer_IOI is defined on the view to handle **insert operations**. When a new record is inserted into vwCustomerAccount, the trigger automatically decomposes the input and executes the appropriate insert statements for both CustomerAuth and CustomerProfile. This design maintains data transparency for end-users while preserving the benefits of vertical partitioning.

2.5.3.2 Indexing Strategy

Indexing is applied strategically to optimise the performance of critical database operations, particularly for the Touch On process. The process involves frequent lookups on card validation, trip history, transaction records, and scanner location data. Composite and Filtered indexes are designed to support common query patterns and improve data retrieval efficiency:

Table	Indexed Columns	Purpose
Trip	(card_id, touch_on_time) INCLUDE (touch_off_time)	Find the newest record in Trip
Trip	(card_id, touch_on_time DESC) WHERE touch_off_time IS NULL	Used to check if this touch action is touch-on or off
Trip	(card_id, touch_on_time DESC) WHERE touch_off_time IS NOT NULL	Used to check if the user touch off within 2 hours
MykiCard	card_id INCLUDE (pass_id, balance, status, daily_cap, card_type)	Check frequently accessed attributes
VehicleStopLog	vehicle_id, stop_station_id, time_stamp DESC	Check if the same vehicle's stop_station_id is changed
DeviceLocation	vehicle_id	Used to update current_stop_station_id from VehicleStopLog
VehicleRealTimeLog	vehicle_id, timestamp	Find the recent logs of a certain vehicle

2.6 Data Dictionary

A comprehensive data dictionary has been compiled to document all tables, columns, data types, constraints, and semantic meanings defined within the *myki* schema.

This dictionary serves as a central reference for developers, testers, and maintainers, and provides detailed metadata for understanding the structure and purpose of each data element in the system.

The table definitions are listed in the **order in which they appear in the DDL script**, to reflect the actual execution sequence and dependencies (e.g., lookup tables defined before referencing tables).

The data dictionary includes the following information for each column:

- **Table:** The name of the table to which the column belongs.
- **Column:** The column name as implemented in the schema.
- **Data Type:** The SQL Server data type assigned to the column.
- **Nullable:** Indicates whether the column allows NULL values.
- **PK:** Indicates whether the column is part of the table's primary key.
- **FK:** Indicates whether the column is a foreign key referencing another table.

- **Description:** A brief description of the column's constraints, purpose, or the data it holds.

The full data dictionary is provided in **Appendix A – Data Dictionary**.

2.7 Special Instructions

The following instructions are provided to support the generation, testing, and operational use of the *MykiTransportDB* database within the educational SQL Server environment.

2.7.1 Database Setup

All database objects are created using standard .sql files, executed in sequence via **Azure Data Studio** or **SQL Server Management Studio**.

The recommended execution order is as follows:

1. **VerticalPartition.sql**
Creates the vertically partitioned tables CustomerAuth and CustomerProfile, along with the associated view vwCustomerAccount and insert trigger trg_vwCustomer_IOI. This script is executed first as it has no external dependencies.
2. **HorizontalPartitions.sql**
Defines all partition schemes (ps_Monthly, ps_Yearly) and corresponding partition functions. This must be executed before any tables that rely on partitioning.
3. **MykiTransportDB_DDL.sql**
Creates all remaining tables and constraints. Partitioned tables are declared with their associated partition schemes during creation. The table creation order is pre-defined within the script to respect foreign key dependencies.
4. **Index.sql**
Applies all necessary clustered and non-clustered indexes to optimise performance for core operations such as touch-on, transaction logging, and trip history retrieval.

2.7.2 Data Entry Guidelines

1. Sample data is populated using a series of seed scripts named sequentially (e.g., 01_seed_CardType.sql, 02_seed_Calendar.sql, etc.). These should be executed in numeric order to respect data dependencies.
2. The Calendar table must include entries for all testing date ranges, with appropriate weekday/weekend/holiday flags. This data is critical for correct fare calculation.
3. Data in MykiCard and MykiPass should be consistent with active or expired status to properly trigger fare logic during touch-on operations.
4. Entities related to Scanner, DeviceLocation, and Vehicle are **fictitious** and do not correspond to real-world Melbourne geography or operational layouts. These are designed purely for simulation and testing purposes.

2.7.3 Testing Procedures

1. Prior to testing, ensure that the database has been populated with appropriate seed data for core entities such as MykiCard, Scanner, DeviceLocation, Calendar, etc.
2. To simulate touch-on functionality, users can manually invoke the stored procedure `usp_TouchOn()` by supplying a valid `card_id` and a `scanner_id` representing the scanner being used.
This simulates a real-world passenger interaction where the Myki card is physically tapped on a scanner. Then, the scanner reads the card's ID and transmits both the `card_id` and its own `scanner_id` to the backend for processing.
3. While no physical hardware interface is implemented in this academic environment, the system is designed with this process in mind. Detailed figure and explanation in Appendix B outline the complete touch-on/off decision flow, covering conditions such as pass validity, fare window eligibility, and Myki Money balance checks.
4. The corresponding T-SQL implementation of this logic is provided in the file `touchOn.sql`.

3.0 DATABASE ADMINISTRATIVE INFORMATION

DATABASE ADMINISTRATIVE INFORMATION

3.1 Responsibility

As this is an academic coursework project, there is no formal organisation responsible for the production operation or long-term maintenance of the *MykiTransportDB* system.

All administrative roles, including database design, deployment, testing, and security configuration, have been carried out by the project developer:

- **Name:** Jordan Chiou
- **Role:** Student Developer & Database Administrator

3.2 System Information

This section documents the technical environment used for implementing and testing the *MykiTransportDB* database system.

3.2.1 Database Management System (DBMS) Configuration

- **Vendor:** Microsoft
- **DBMS:** Microsoft SQL Server 2022 (Developer Edition)
- **Platform:** Installed locally via SQL Server Management Studio (SSMS) and Azure Data Studio
- **Environment:** Educational / instructional use only

The DBMS was configured to run on a single-instance local deployment under the default configuration.

Partitioning, indexing, constraints, and stored procedures were implemented using T-SQL.

Limitations:

- No distributed processing features (e.g., replication, clustering) were enabled.
- Custom filegroups, file-level I/O balancing, and advanced security (e.g., encryption) were not used due to restrictions in the academic version.

3.2.2 Hardware Configuration

- **Host Machine:** Apple MacBook (running macOS 12.7 Monterey)
- **Virtual Environment:** Microsoft SQL Server executed via Docker container or local virtual machine with Windows-based SQL Server environment

- **Processor:** Apple M1 / M2 Chip
- **Memory:** 16GB RAM
- **Disk Storage:** Local SSD (~500GB capacity)

The database system was tested in a simulated local environment suitable for academic use. No dedicated server or cloud infrastructure was used for deployment.

3.2.3 Database Software Utilities

The following DBMS utility software was used to support the design, creation, and maintenance of the *MykiTransportDB* database:

- **SQL Server Management Studio (SSMS)**
 - Version: 19.x (Developer Edition)
 - Purpose: GUI-based environment for executing T-SQL queries, managing schema objects, monitoring indexes, and viewing execution plans.
- **Azure Data Studio**
 - Version: 1.46+
 - Purpose: Lightweight cross-platform SQL editor used for managing .sql files, executing DDL/DML scripts, and visualising database diagrams.
- **SQLCMD (optional use)**
 - Used for executing scripts via command line in batch processing mode during testing.

Official documentation for Microsoft SQL Server and its management utilities can be found at: <https://learn.microsoft.com/en-us/sql/>

3.2.4 Support Software Available for Maintaining Database

The database was developed and tested within an academic environment using the following support software and platforms:

Software	Version	Function	Characteristics
Microsoft SQL Server 2022	Developer Edition	DBMS	Full T-SQL support, indexing, partitioning, stored procedures
macOS Monterey	12.7	Host OS	Local development platform
Windows Server Core (via Docker / VM)	N/A	SQL Server runtime	Runs SQL Server instance in isolated environment
SSMS	19.x	DBMS utility	Object management, execution plans, backups
Azure Data Studio	1.46+	Lightweight SQL IDE	Script authoring and visualisation

3.2.5 Security

No formal user-level access control is implemented in this academic deployment. All database objects reside under the default schema `dbo`, and all actions are executed under a trusted developer account.

In the current version:

- All data integrity constraints are enforced through **primary key**, **foreign key**, and **check constraints**
- No encryption, row-level security, or audit logging is implemented due to project scope and platform limitations

3.3 Storage Requirements

Storage estimation for the *MykiTransportDB* database involves multiple components, including row-level storage calculations, partition-based volume growth, and optional indexing overhead.

Due to the complexity and length of these calculations, including formula derivations, projected growth over 10 years, and storage impacts across multiple partitioned tables, a detailed breakdown has been provided in **Appendix C – Storage Requirement Analysis**.

3.4 Recovery

In the event of data loss, corruption, or system failure, the *MykiTransportDB* database can be re-established using the provided schema and configuration scripts.

The recovery process is designed to be **deterministic and reproducible**. A series of structured `.sql` files defines all schema components, including partition schemes, table definitions, indexes, views, and triggers.

All database scripts are **version-controlled** via Git and are publicly available on GitHub:

🔗 <https://github.com/jordan0726/MyProject/tree/master/Database/MykiSystemDesign>

This ensures that each schema revision is fully documented and can be restored or rolled back as needed for maintenance, testing, or deployment purposes.

3.4.1 Recovery Methodology

The following steps outline the recovery procedure for reinitialising the database:

1. **Recreate the Database Instance**

A new SQL Server database instance should be created using the default schema (`dbo`) with the same name or logical reference (*MykiTransportDB*).

2. **Reapply Partition Schemes and Functions**

Run `HorizontalPartitions.sql` to re-establish all required partition schemes (`ps_Monthly`, `ps_Yearly`) and partition functions used by time-series tables.

3. Recreate Vertically Partitioned Tables and Views

Execute VerticalPartition.sql to create CustomerAuth, CustomerProfile, and the associated view vwCustomerAccount with its insert trigger trg_vwCustomer_IOI.

4. Recreate Main Schema Objects

Run MykiTransportDB_DDL.sql to create all entity tables, reference tables, constraints, and check conditions. This script also defines partitioned tables with proper storage mapping.

5. Reapply Indexes

Execute Index.sql to create all performance-related indexes, including composite and non-clustered indexes optimised for query efficiency.

6. Restore Data (Optional)

If necessary, restore seed data using the seedXX.sql scripts in order from 01_ onward. This repopulates lookup tables, test cards, and initial calendar data for simulation.

7. Verify Logical Access

Confirm view and procedure accessibility (e.g., test usp_TouchOn) to ensure the system is fully functional and the schema is logically consistent.

3.4.2 Notes

- All recovery operations assume access to a working SQL Server environment with permissions to execute DDL and DML scripts.
- If only schema needs recovery (not data), steps 1–5 are sufficient.
- Scripts should be executed in the prescribed order to preserve foreign key and partition dependencies.

3.5 Partition/File Information**3.5.1 Content**

Partitioning is applied to specific high-volume tables to enhance scalability and optimise query performance for time-based operational data. Each table is partitioned based on a DATETIME2 column that aligns with the data's lifecycle and access pattern.

The following tables are logically partitioned using either monthly or yearly range-based strategies:

Table	Partition Key	Partition Strategy	Description
Trip	touch_off_time	Monthly Range	Records all completed MykiCard journeys upon tap-off
CardTransaction	timestamp	Monthly Range	Logs fare deductions, top-ups, and pass purchases
VehicleStopLog	update_timestamp	Monthly Range	Captures vehicle arrivals/departures

			at stops
VehicleRealTimeLog	log_timestamp	Monthly Range	Stores GPS-based movement data for vehicles
MykiPass	expiry_date	Yearly Range	Manages long-term pass validity and lifecycle

Each monthly partition contains data corresponding to one calendar month based on the partition key's value. For example, all transactions with timestamp values falling within **March 2024** are stored in the partition mapped to that month's range.

Yearly partitioning is applied to the MykiPass table to support long-term archival and validity checks for time-bound transit passes.

3.5.2 Description

Partitioning in *MykiTransportDB* is implemented using **SQL Server partition functions and schemes**, and is designed to segment time-series data for better manageability and performance.

- **Partition Function (Monthly):** pf_Monthly
 - Type: RANGE RIGHT
 - Generated using a recursive T-SQL common table expression (CTE) and STRING_AGG() to build monthly boundary values dynamically
 - Range covers the period from 2024-02-01 (inclusive) to 2030-01-01 (exclusive), with one partition per calendar month
- **Partition Scheme:** ps_Monthly
 - All partitions are mapped to the default filegroup PRIMARY using ALL TO ([PRIMARY])
- **Partition Function (Yearly):** pf_Yearly
 - Type: RANGE RIGHT
 - Manually defined boundaries from 2025-01-01 to 2035-01-01, incremented annually
 - Used by the MykiPass table to support coarse-grained archival and pass expiry evaluation
- **Partition Scheme:** ps_Yearly
 - Similarly maps all partitions to the default PRIMARY filegroup
- **Design Characteristics:**

- No physical sharding or multi-filegroup distribution is used due to academic SQL Server restrictions
- Partitioning is logical only, and primarily aids in **partition elimination** during queries on datetime-based keys (e.g., touch_off_time, timestamp)
- The partitioned tables share the same logical scheme but are decoupled structurally to preserve flexibility and modularity
- **Filegroup Usage:**
Due to platform constraints in the educational SQL Server environment, all partitions are mapped to the default PRIMARY filegroup. No custom filegroups or file-level distribution strategies (e.g., placing each partition on a different physical volume) have been implemented.

The current configuration ensures simplicity and compatibility with local development tools, while maintaining logical separation through partition schemes and functions.

3.5.3 Partition/File Interdependencies

While each partitioned table in *MykiTransportDB* is implemented independently, several **logical interdependencies** exist between them, primarily due to **time-based operations** and **data lineage relationships**. These interdependencies are designed to support cohesive query access and maintain alignment of operational workflows:

3.5.3.1 Temporal Alignment Between Operational Tables

- The Trip and CardTransaction tables both use **monthly partitioning** (ps_Monthly) on DateTime fields that are **sequentially related**:
 - Trip.touch_off_time indicates trip completion
 - CardTransaction.timestamp records the corresponding fare deduction

Aligning these partitions improves **join performance** and ensures that related trip and transaction data can be efficiently queried within the same partition boundary.

3.5.3.2 Real-Time Vehicle Data Relationships

- VehicleStopLog and VehicleRealTimeLog are both partitioned monthly using update_timestamp and log_timestamp, respectively. While they do not share a direct key relationship, their **data generation is sequential and correlated**.

Partitioning them on the same temporal scale enables efficient analytic queries and reduces join overhead during operational diagnostics or transit monitoring.

3.5.3.3 Foreign Key Partition Alignment

- The CardTransaction table contains a **foreign key reference** to Trip. Since SQL Server requires that partitioned tables involved in foreign key constraints have matching partition key alignment, touch_off_time is included in both tables to support referential integrity across partitions.

3.5.3.4 Yearly Partitioned Archival Table

- MykiPass is the only table partitioned by **yearly expiry date**, under ps_Yearly. It does not participate in direct joins with other partitioned tables and is used primarily for long-term validity management. This allows it to remain **logically and physically independent**, supporting coarse-grained access and potential archival in future versions.

3.6 Database Interfaces

No external database interfaces are implemented in the current academic version of *MykiTransportDB*.

The system operates as a **self-contained backend database** used for demonstrating schema design, partitioning strategy, and fare-processing logic in a standalone SQL Server environment:

3.6.1 Description of Operational Implications

Since no data is exchanged across external systems, no immediate operational implications are related to data transfer or remote access.

3.6.2 Description of Data Transfer Requirements

The current system does not require formal data transfer mechanisms. All test data is loaded manually via .sql seed scripts.

3.6.3 Description of Formats of Data

All data is currently stored and accessed in native **SQL Server data formats** (e.g., INT, VARCHAR, DATETIME2). No format translation or conversion is needed at this stage.

3.7 Error Handling

Error handling within the *MykiTransportDB* system is implemented at the **stored procedure level**, using SQL Server's native control-of-flow and exception-handling mechanisms.

The key procedures responsible for core business logic (usp_TouchOn, usp_CheckTapCardStatus, usp_ProcessTouchOffTransaction, etc.) include **validation checks** and structured error responses to ensure data consistency and prevent invalid operations.

Appendix A – Data Dictionary

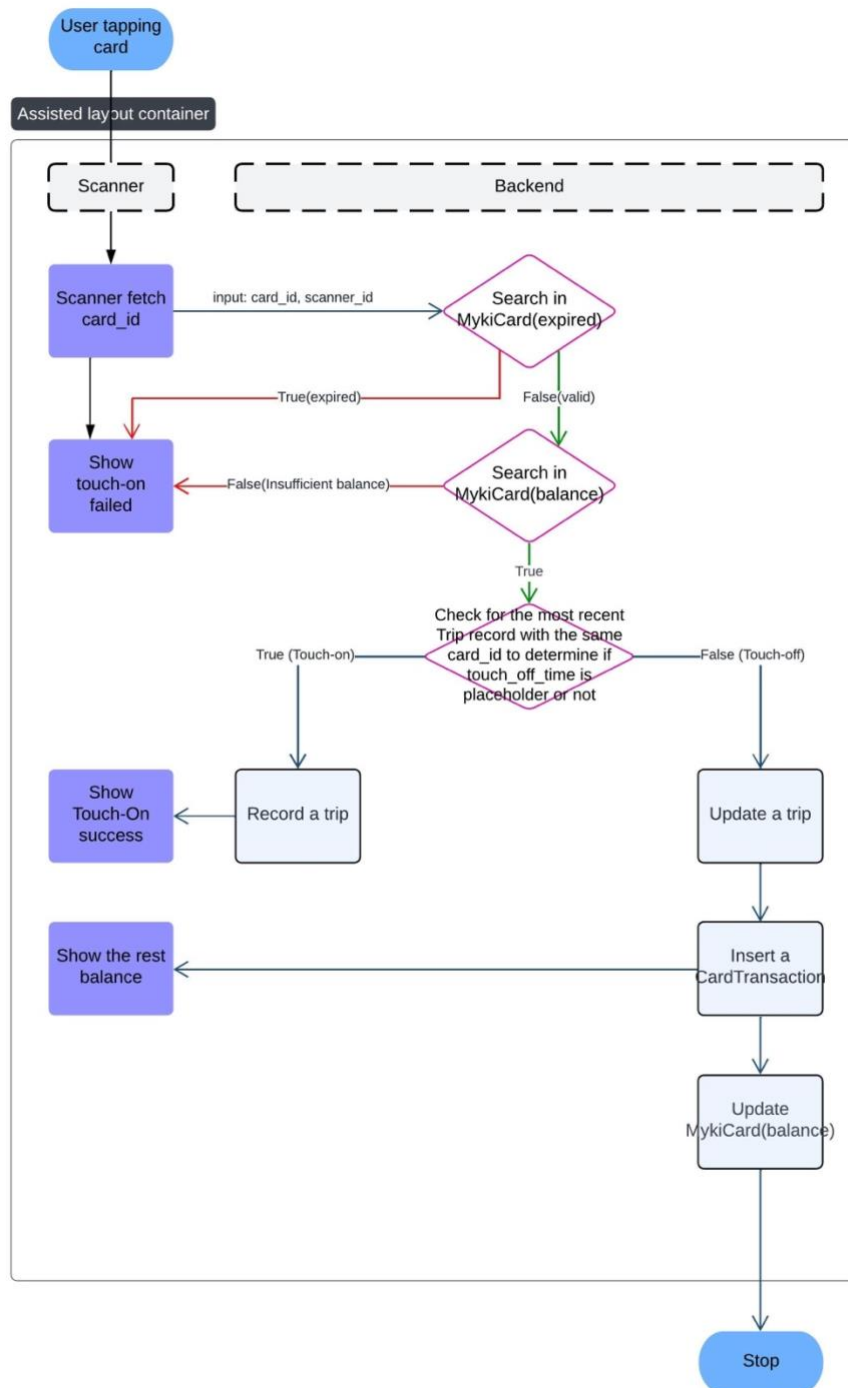
Table	Column	Data Type	Nulla ble	P K	F K	Description
CardType	card_type	VARCHAR (20)	NO	✓		Primary key for card type
CardType	description	VARCHAR (100)	YES			Description for the card type
FareType	fare_type	VARCHAR (20)	NO	✓		Primary key for fare type
FareType	description	VARCHAR (100)	YES			Description of fare type (e.g. default_fare, concession)
MykiPass	pass_id	INT	NO	✓		Primary key with identity(1,1)
MykiPass	start_date	DATE	NO			Start date of the pass
MykiPass	expiry_date	DATE	NO	✓		Expiry date of the pass, part of PK due to the Monthly partition
MykiPass	expired	BIT	YES			Indicates if the pass is expired (default 0)
MykiCard	card_id	INT	NO	✓		Unique ID for each card, auto-incremented
MykiCard	balance	DECIMAL(10,2)	NO			Current card balance, must be >= -10
MykiCard	customer_id	INT	YES		✓	Customer who owns the card, links to CustomerAuth
MykiCard	card_type	VARCHAR (20)	NO		✓	Type of the card (e.g., full-fare, concession), references CardType
MykiCard	pass_id	INT	YES		✓	Part of foreign key to MykiPass, identifies the pass assigned to this card
MykiCard	pass_expiry_date	DATE	YES		✓	Pass expiration date, used with pass_id as composite FK
MykiCard	daily_cap	DECIMAL(4,2)	YES			Maximum fare charged in a day before capping applies (default 0)
MykiCard	status	BIT	YES			Indicates whether the card is active (1 = active, 0 = inactive)
MykiCard	expiry_date	DATE	YES			Date the card itself expires
DeviceLocation	device_location_id	INT	NO	✓		Unique ID for each device location
DeviceLocation	vehicle_id	INT	YES			Vehicle the scanner is attached to (if applicable). It is mutually exclusive with stop_station_id
DeviceLocation	stop_station_id	INT	YES			Stop station the scanner is installed at (if applicable). It is mutually exclusive with vehicle_id
DeviceLocation	location_type	VARCHAR (20)	NO			Describes the type of location (e.g. 'physical', 'online')
DeviceLocation	current_stop_station_id	INT	YES			The stop station currently associated with this device. Auto updated from VehicleStopLog

Scanner	scanner_id	INT	NO	✓	Unique ID for each scanner device
Scanner	device_location_id	INT	NO	✓	References the location where the scanner is installed
Trip	trip_id	INT	NO	✓	Unique ID for the trip
Trip	card_id	INT	NO	✓	The card used for the trip, references MykiCard
Trip	touch_on_time	DATETIME2(0)	NO		The time the card was touched on
Trip	touch_on_scanner_id	INT	YES	✓	Scanner used to touch on, references Scanner
Trip	touch_on_stop_station_id	INT	YES		Stop station where the trip started. System fetches from DeviceLocation(current_stop_station_id) when executing udf_GetCurrentStopStationIdByScannerId.
Trip	touch_off_time	DATETIME2(0)	YES	✓	The time the card was touched off, also used for monthly partitioning
Trip	touch_off_scanner_id	INT	YES	✓	Scanner used to touch off, references Scanner
Trip	touch_off_stop_station_id	INT	YES		Stop station where the trip ended
Trip	fare_type	VARCHAR(20)	YES	✓	Type of fare applied (e.g. default, free_2hours). Calculated by usp_DetermineFareType.
CardTransaction	transaction_id	INT	NO	✓	Unique ID for each transaction
CardTransaction	card_id	INT	NO	✓	Card involved in the transaction, references MykiCard
CardTransaction	trip_id	INT	YES	✓	Trip associated with the transaction, part of composite FK to Trip
CardTransaction	touch_off_time	DATETIME2(0)	YES		Time of touch off, used with trip_id as composite FK
CardTransaction	scanner_id	INT	YES	✓	Scanner used for the transaction
CardTransaction	amount	DECIMAL(10,2)	NO		Transaction amount, must be >= 0
CardTransaction	timestamp	DATETIME2(0)	NO	✓	When the transaction was recorded, used for monthly partition
CardTransaction	transaction_type	VARCHAR(20)	NO		Type of transaction: deduction, top-up, refund, or free
Calendar	calendar_date	DATE	NO	✓	Date used as primary key, yearly partitioned
Calendar	is_weekend	BIT	NO		1 if weekend, 0 otherwise
Calendar	is_holiday	BIT	NO		1 if it's a public holiday, 0 otherwise
Calendar	holiday_name	VARCHAR(100)	YES		Name of the holiday if applicable
Calendar	day_of_week	VARCHAR(10)	NO		Day of the week, e.g., Monday

Calendar	year	INT	NO			Calendar year, must be >= 2000
Calendar	month	INT	NO			Calendar month, between 1 and 12
Calendar	day_of_month	INT	NO			Day of the month, between 1 and 31
StopStation	stop_station_id	INT	NO	✓		Unique ID for stop station
StopStation	station_name	VARCHAR(100)	NO			Name of the station
StopStation	zone	VARCHAR(50)	YES			Fare zone of the station (e.g. zone1, zone2 or the overlapping area -> zone1+zone2)
StopStation	geo_point	GEOGRAPHY	YES			Geographic coordinates of the station
Route	route_id	INT	NO	✓		Unique ID for each route
Route	route_name	VARCHAR(100)	NO			Name of the route
Route	start_time_of_day	TIME	NO			Scheduled departure time
Route	last_run_time	DATETIME2(0)	YES			Last recorded run time (if available)
RouteStop	route_id	INT	NO	✓	✓	Route ID, part of the composite primary key
RouteStop	stop_station_id	INT	NO	✓	✓	Station ID, part of the composite primary key
RouteStop	direction	VARCHAR(20)	NO	✓		Direction of travel (inbound/outbound), part of composite PK
RouteStop	scheduled_arrive_time	DATETIME2(0)	YES			Expected arrival time at the stop
VehicleType	vehicle_type	VARCHAR(50)	NO	✓		Code for type of vehicle (e.g., tram, bus)
VehicleType	description	VARCHAR(200)	YES			Optional description for the vehicle type
Vehicle	vehicle_id	INT	NO	✓		Unique ID for each vehicle
Vehicle	vehicle_type	VARCHAR(50)	NO		✓	References the vehicle type from VehicleType
VehicleRun	run_id	INT	NO	✓		Unique ID for each scheduled or real-time vehicle run
VehicleRun	vehicle_id	INT	NO		✓	References the vehicle making the run
VehicleRun	route_id	INT	NO		✓	References the route being followed
VehicleRun	direction	VARCHAR(20)	NO			Direction of the run, either 'inbound' or 'outbound'
VehicleRun	start_time	DATETIME2(0)	NO			Start time of the vehicle run
VehicleRealTimeLog	vehicle_realtime_log_id	INT	NO	✓		Unique ID for each real-time GPS log
VehicleRealTimeLog	vehicle_id	INT	NO		✓	Vehicle sending the GPS data
VehicleRealTimeLog	latitude	DECIMAL(9,6)	NO			Latitude position of the vehicle
VehicleRealTimeLog	longitude	DECIMAL(9,6)	NO			Longitude position of the vehicle

VehicleRealTimeLog	speed	DECIMAL(10,2)	YES			Current speed of the vehicle
VehicleRealTimeLog	log_timestamp	DATETIME2(0)	NO	✓		Timestamp of log entry, used for monthly partitioning
VehicleStopLog	vehicle_stop_log_id	INT	NO	✓		Unique ID for each stop log entry
VehicleStopLog	vehicle_id	INT	NO		✓	Vehicle that generated the stop log
VehicleStopLog	stop_station_id	INT	NO		✓	The station the vehicle was nearest to. Auto calculated by the system
VehicleStopLog	update_timestamp	DATETIME2(0)	NO	✓		Timestamp of the stop event, used for monthly partitioning

Appendix B – TouchOn Process



Touch-On Stored Procedure Logic (usp_TouchOn)

The stored procedure usp_TouchOn handles the passenger card tap-in process and must be called with two input parameters: card_id and scanner_id.

The execution flow is as follows:

1. **Card Status Validation**

The procedure begins by invoking usp_CheckTapCardStatus, verifying whether the provided card has expired or a negative balance.

- The transaction is immediately terminated if the card is invalid (e.g., expired or insufficient funds).
- No records are written to the Trip or CardTransaction tables, and an error message indicating the issue is returned.

2. **Determine Tap Type (Touch-On or Touch-Off)**

The procedure then queries the Trip table to find the most recent trip record associated with the same card_id.

- If the touch_off_time of that record is still a placeholder (i.e., NULL or a predefined default), the system treats the current tap as a **touch-off**.
- Otherwise, the tap is considered a **touch-on**.

3. **Processing Touch-On**

If the tap is determined to be a **touch-on**, the system calls the helper procedure usp_InsertTripOnTouchOn, creating a new entry in the Trip table to record the journey's beginning.

4. **Processing Touch-Off**

If the tap is recognised as a **touch-off**, the system calls usp_ProcessTouchOffTransaction. This procedure:

- Completes the corresponding Trip entry by setting touch_off_time and related metadata.
- Calculates the fare and inserts a corresponding record into the CardTransaction table.
- Updates the card's balance and daily_cap fields accordingly.

5. **Response to Scanner**

After successful execution, the procedure returns a message to the calling system (simulated scanner) indicating:

- "Touch-On Success" or "Touch-Off Success"
- The updated remaining balance can then be displayed on the scanner interface.

Appendix C - Storage Requirement Analysis

1. Annual Record Count Projections Based on Ridership

According to the 2023–2024 PTV Annual Report, metropolitan services in Victoria carry the following passenger volumes:

- Buses: 114.9 million
- Trains: 182.5 million
- Trams: 154.8 million

This amounts to approximately 452.2 million passenger trips annually. For estimation purposes, each passenger trip is assumed to correspond to a single **Trip** record in the database (i.e., each tap-on/tap-off event generates one record). Although real-world scenarios often involve multiple taps within a single journey, this assumption serves as a reasonable baseline.

2. SQL Server Data Type Storage Basics

The MykiTransportDB schema is implemented using Microsoft SQL Server, where efficient storage estimation is critical for capacity planning and optimisation. The table below summarises the storage requirements for the primary data types used across the database schema.

Data Type	Typical Storage Size	Notes
INT	4 bytes	Fixed size for whole numbers.
DECIMAL(p, s)	Varies based on precision and scale. For example, DECIMAL(10,2) requires 5 bytes.	Commonly used for monetary values.
DATE	3 bytes	Represents dates without time.
DATETIME2(0)	6 bytes	Precise to the nearest second (0 fractional seconds).
TIME	3–5 bytes (depending on precision)	TIME(0) uses 3 bytes.
BIT	1 byte (per up to 8 columns)	Compact boolean flag.
VARCHAR(n)	Variable: up to n bytes + 2 bytes overhead	Actual storage depends on character count. For example, VARCHAR(50) with a 36-character value uses 38 bytes.
GEOGRAPHY	~22–32 bytes (depends on complexity)	Used for storing spatial coordinates.

In addition to raw data type storage, each row includes **row-level metadata** (e.g. row headers, column offset arrays) and **index overhead** (e.g. clustered and non-clustered index pages). These factors vary depending on schema design, page fill factors, and indexing strategy.

The storage estimation for each table will be calculated in subsequent sections using these baseline values, incorporating assumptions regarding average field usage (e.g. average string lengths for VARCHAR) and record frequency.

3. Example Calculation: Trip Table

3-1. Table Structure

Column	Data Type	Notes
trip_id (PK)	VARCHAR(50)	Primary Key (e.g., GUID or other unique string)
card_id (FK)	VARCHAR(50)	Foreign key referencing MykiCard
touch_on_time	DATETIME	Datetime of tap-on
touch_on_scanner_id	VARCHAR(50)	Scanner used for tap-on
touch_on_stop_station_id	VARCHAR(50)	Stop/station for tap-on
touch_off_time (nullable)	DATETIME	Datetime of tap-off (null if not tapped off)
touch_off_scanner_id (nullable)	VARCHAR(50)	Scanner used for tap-off (if applicable)
touch_off_stop_station_id	VARCHAR(50)	Stop/station for tap-off (if applicable)
fare_charged	DECIMAL(10,2)	Fare amount charged
fare_type	VARCHAR(20)	Type of fare (e.g., full fare, concession)

3-2. Estimated Size for Trip Table

Assuming an average string length of ~36 characters for VARCHAR(50) columns plus overhead:

Column	Data Type	Size (Bytes)	Notes
trip_id	INT	4	Identity column (auto-increment)
card_id	INT	4	Foreign key to MykiCard
touch_on_time	DATETIME2(0)	6	Precision to the second
touch_on_scanner_id	INT	4	
touch_on_stop_station_id	INT	4	
touch_off_time	DATETIME2(0)	6	May be NULL for incomplete trips
touch_off_scanner_id	INT	4	
touch_off_stop_station_id	INT	4	
fare_type	VARCHAR(20)	14	Assuming average 12 characters stored

A preliminary summation of the Trip table fields results in approximately **50 bytes** of storage per record, based on the following breakdown:

- 6 integer fields (INT): 6×4 bytes = 24 bytes
- 2 datetime fields (DATETIME2(0)): 2×6 bytes = 12 bytes
- 1 short string field (VARCHAR(20)): estimated average of 12 characters + 2 bytes overhead $\approx$ 14 bytes

To account for SQL Server's internal structures, an additional **8 bytes** is allocated for row overhead (e.g. row header and null bitmap), and **30 bytes** for indexing and metadata. This results in an estimated total of:

50 bytes (data fields) + 8 bytes (row overhead) + 30 bytes (indexing) $\approx$ 88 bytes per Trip record

With an estimated **452.2 million passenger trips in 2023**, the projected annual storage requirement for the Trip table is:

$$452,200,000 \text{ records/year} \times 88 \text{ bytes/record} \approx 39,793,600,000 \text{ bytes} \approx 39.8 \text{ GB/year}$$

This estimation assumes a fully completed trip (i.e. both tap-on and tap-off events are recorded), and does not account for compression or advanced indexing strategies, which may reduce or increase the final storage consumption depending on implementation.

3-3. Estimated Size for Indexes on Trip

There are three indexes based on the Trip table: IX_Trip_Card_TouchOn, IX_Trip_UnfinishedTripCheck, and IX_Trip_CompletedTripLookup.

The simplified estimated yearly storage as below:

$$\text{IX_Trip_Card_TouchOn} \rightarrow 452,200,000 \times 24 \text{ bytes} \approx 10.8 \text{ GB}$$

$$\text{IX_Trip_UnfinishedTripCheck} \rightarrow 13,566,000 \times 18 \text{ bytes} \approx 244.2 \text{ MB } (\sim 0.24 \text{ GB})$$

$$\text{IX_Trip_CompletedTripLookup} \rightarrow 438,634,000 \times 18 \text{ bytes} \approx 7.9 \text{ GB}$$

3-4. Conclusion

While each Trip record occupies **~88 bytes**, the **total index overhead adds approximately 19 GB/year**, bringing the total annual storage for the Trip table to:

$$39.8 \text{ GB (data)} + 18.9 \text{ GB (indexes)} \approx \text{“58.7 GB/year”}$$

4. Example Calculation: Transaction Table

A preliminary estimation of the CardTransaction table fields results in the following storage breakdown:

4.1 Table Structure:

Column	Data Type	Estimated Size (Bytes)	Notes
transaction_id	INT	4	Identity column
card_id	INT	4	Foreign key to MykiCard
trip_id	INT (nullable)	4	Optional foreign key
touch_off_time	DATETIME2(0)	6	Time of trip completion
scanner_id	INT (nullable)	4	Optional
amount	DECIMAL(10,2)	5	Fare or monetary value
timestamp	DATETIME2(0)	6	Creation time of transaction
transaction_type	VARCHAR(20)	14	Average length ~12 characters (e.g. 'top-up')

4.2 Estimated Size per Transaction Record

Estimated total per record:

$$47 \text{ (data)} + 8 \text{ (row)} + 30 \text{ (index)} \approx 85 \text{ bytes}$$

Assuming **80%** of the **452.2 million** trips result in a transaction (based on deduction/top-up logic):

$$361,760,000 \times 85 \text{ bytes} \approx 30,749,600,000 \text{ bytes} \approx 30.75 \text{ GB/year}$$

5. Summary of Storage Calculation for 2023

The following table consolidates the rough annual (or total) storage estimates for the primary tables, based on earlier capacity calculations. Detailed assumptions and step-by-step derivations are similar to the two examples above:

Table Name	Estimated Record Count (2023)	Approx. Size per Record	Estimated Storage (Annual / Static)
Trip	~452.2 million	~88 bytes	~39.8 GB/year
CardTransaction	~361.76 million	~85 bytes	~30.75 GB/year
VehicleStopLog	~306.6 million/year	~32 bytes	~9.8 GB/year
VehicleRealTimeLog	~306.6 million/year	~40 bytes	~12.3 GB/year
MykiCard	~2.88 million	~80 bytes	~230 MB (static)
MykiPass	~288,000	~50 bytes	~14 MB (static)
CustomerAuth	~2.4 million	~160 bytes	~384 MB (static)
CustomerProfile	~2.4 million	~180 bytes	~432 MB (static)

Annual tables:

Trip + Transaction + VehicleStopLog + VehicleRealTimeLog
 = 39.8 + 30.75 + 9.8 + 12.3 = **~92.65 GB/year**

Static profile tables:

MykiCard + MykiPass + CustomerAuth + CustomerProfile
 = 0.23 + 0.014 + 0.384 + 0.432 = **~1.06 GB**

Grand Total Estimated Storage (2023)

≈ **92.65 GB/year** (growing tables)
 + **1.06 GB** (static profile data)
 = **~ 93.7 GB total in 2023**

6. Projection for next 10 years

The Victorian population is forecast to grow from **6.8 million in 2023 to 10.3 million in 2051**. To translate this demographic trend into database growth, we proceed in four steps:

1. Annualised population growth rate: ~ 1.49% per year
2. 2033 population estimate= $6.8\text{M} \times (1.0149)^{10} \approx 7.89\text{million}$
3. Trip volume projection:
 2023 produced **452.2 million** passenger trips ⇒ **66.5 trips / capita**.
 Assuming the same per-capita demand in 2033:
Trips₂₀₃₃ = 7.89M × 66.5 ≈ 524.2 million trips / year
4. Storage projection (key growing tables)

Table	Size / row	2023 rows	2023 GB	2033 rows (↑15.9 %)	2033 GB
Trip	88 B	452.2 M	39.8 GB	524.2 M	46.1 GB

CardTransaction	85 B	361.8 M	30.8 GB	419.4 M	35.6 GB
-----------------	------	---------	---------	---------	---------

Other high-volume logs are driven by fleet telemetry, not ridership, and are assumed constant to 2033:

Log Table	Rows/yr	Size/row	Storage
VehicleStopLog	32 B	306.6 M	9.8 GB
VehicleRealTimeLog	40 B	306.6 M	12.3 GB

Static or slowly growing registries scale with population; applying the same 15.9 % uplift:

Table	2023 GB	2033GB
MykiCard	0.23	0.27
MykiPass	0.014	0.016
CustomerAuth	0.384	0.445
CustomerProfile	0.432	0.500

Victoria's projected compound annual population growth of ~1.5 % will increase the resident base from 6.8 million in 2023 to ~7.9 million by 2033—up 15.9 %. If trip-making rates stay constant (≈ 66.5 trips per capita), annual journeys will grow from 452 million to roughly 524 million, affecting the ride-driven tables: Trip and CardTransaction. Their combined footprint will increase by just over 12 GB, while vehicle telemetry logs remain stable, and static account tables grow marginally with new cardholders. Thus, the total annual storage requirement will rise from ~93 GB in 2023 to ≈ 105 GB by 2033. This moderate increase indicates that the current partitioning and hardware configuration can manage a decade of ridership growth before substantial capacity or archival measures are necessary.